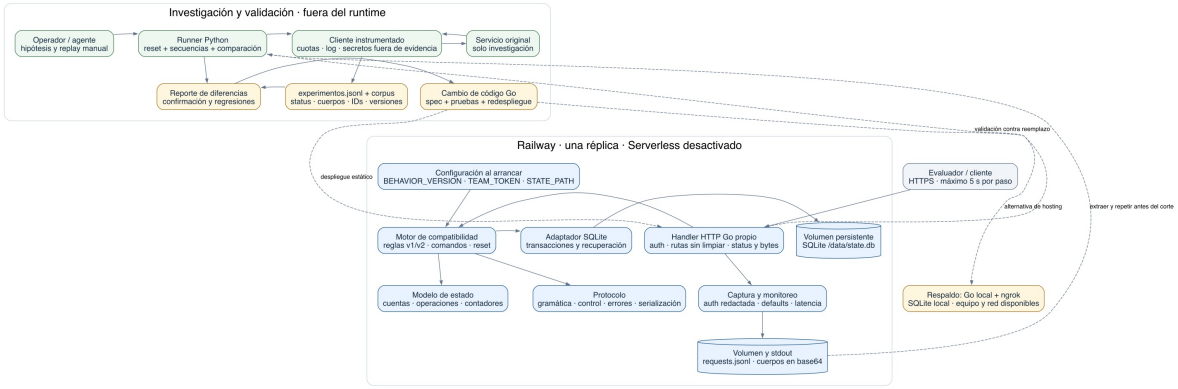
Caja negra: diseño del reemplazo HTTP

Estado: diseño corregido con evidencia `v1`, 10 de octubre de 2026. Implementación pendiente.
Objetivo: reproducir el servicio vigente a la entrega, incluidos errores y memoria entre requests. El reemplazo es autónomo, responde en menos de 5 segundos por paso y conserva su estado tras reinicios. La especificación incremental queda separada de las decisiones de arquitectura; las observaciones de `v1` deben revalidarse tras el cambio.

Arquitectura y decisiones



Arquitectura del reemplazo y de la investigación

Fuente editable: architecture.mmd. PNG. Coordinación: PROGRESS.md. Contrato: enunciado. Plan experimental: estrategia de descubrimiento.

Decisión	Razón y límite
Go con `net/http`, un proceso y una réplica	Experiencia del equipo, ejecución determinista y operación sencilla. No escalar horizontalmente durante el reto.
SQLite en volumen persistente	Mantener cuentas, operaciones y contadores sin un servidor de base de datos. Memoria solo para arranque y pruebas; no para entrega final.
Exploración en Python, fuera del servicio Go	Aprovechar el plan de investigación paralelo. Solo el cliente instrumentado consulta el original; el binario desplegado no contiene ese cliente ni recibe `BB_TOKEN`.
Railway; local + ngrok como respaldo	Una réplica, Serverless desactivado y volumen montado. Comprobar cuenta, créditos y continuidad; el respaldo requiere equipo y red disponibles.
Actualizaciones mediante código y redespliegue	Detectar cambios con replay manual; no autoadaptación ni consultas al original durante requests del evaluador.
Fidelidad antes que mejoras de negocio	Mantener typos, espacios y comportamientos observados. Documentar riesgos sin corregirlos en el reemplazo compatible.
Handler propio, sin `http.ServeMux`	Comparar método y ruta sin limpiar ni redirigir: `GET /msg` y `POST //msg` dan 404 E404 (log 41, 44).
Reglas versionadas por configuración de arranque	`BEHAVIOR_VERSION=v1/v2`; sin valor, última versión implementada. Versión desconocida impide arrancar. Nunca elegir por reloj.

El único camino al original es **operador → herramienta de exploración → cliente instrumentado → original**. El evaluador solo llega al **reemplazo → SQLite**. La evidencia se convierte en reglas mediante revisión y cambios de código.

Capas y dependencias

Capa	Responsabilidad	Dependencias permitidas
Composición (`cmd/replacement`)	Cargar configuración, abrir almacenamiento, conectar componentes y arrancar/cerrar el servidor.	Adaptador HTTP, motor y almacenamiento concreto.
Adaptador HTTP (`internal/httpadapter`)	Capturar método, ruta, autenticación y bytes del cuerpo; escribir status y bytes exactos. Controlar readiness y errores de transporte según evidencia.	Motor y tipos de protocolo. Sin SQL ni reglas de saldos.
Motor de compatibilidad (`internal/engine`)	Coordinar validaciones y comandos en el orden observado, seleccionar respuestas, ejecutar consultas/mutaciones/reset de forma indivisible.	Protocolo, modelo de estado y contrato de almacenamiento. Sin HTTP ni cliente del original.
Protocolo (`internal/protocol`)	Gramática de `m`, control, representación numérica, errores y serialización compatibles. No normalizar ni usar defaults del framework sin evidencia.	Biblioteca estándar. Sin base de datos ni red.
Modelo (`internal/state`)	Tipos de cuentas, operaciones, contadores y cambios de estado; reglas aprendidas, sin inventar restricciones bancarias.	Biblioteca estándar.
Persistencia (`internal/storage`)	Implementar el contrato del motor con transacciones y lecturas del estado, en SQLite o memoria. Persistir resultados completos y recuperar tras reinicio.	Modelo de estado; driver SQLite únicamente en la implementación concreta.
Herramientas (`tools/explorer`)	Hipótesis, sondas, secuencias, registro, replay y diferencias. Ejecución manual independiente del runtime.	Cliente HTTP instrumentado y artefactos; nunca importadas por el servicio Go.

La interfaz de almacenamiento pertenece al consumidor del motor: debe permitir una operación atómica que lea y modifique conjuntamente cuentas, operaciones y contadores. Los adaptadores concretos implementan ese contrato. No dividir una transferencia en escrituras independientes ni fijar todavía la precedencia de validaciones.

Árbol de directorios propuesto

Los directorios de implementación siguientes son **propuestos**, no código ya creado. El cliente `tools/explorer/bb.py` apareció durante el trabajo paralelo; se reutilizará como única vía al original, verificando sus garantías antes de ampliar el runner.

```
.
├── AGENTS.md
├── PROGRESS.md
├── README.md                        # uso, comandos y enlace al diseño
├── go.mod / go.sum                  # dependencias Go, al implementar
├── cmd/
│   └── replacement/main.go          # composición; único binario desplegado
├── internal/
│   ├── httpadapter/                 # rutas, sobre HTTP y escritura exacta
│   ├── engine/                      # orquestación, comandos y contrato Store
│   ├── protocol/                    # parser, control, errores y respuestas
│   ├── state/                       # cuentas, operaciones y contadores
│   └── storage/
│       ├── memory/                  # desarrollo y pruebas
│       └── sqlite/                  # transacciones, esquema y migraciones
├── tools/
│   └── explorer/                    # Python; fuera del despliegue Go
│       ├── cli.py                   # baseline, replay y comparación
│       ├── bb.py                    # cliente existente; log y cuotas
│       ├── sequences.py             # reset y ejecución ordenada
│       ├── probes/                  # hipótesis, fronteras y fuzzing
│       └── compare.py               # semántica JSON y bytes/texto exactos
├── tests/
│   ├── corpus/                      # secuencias confirmadas y versionadas
│   └── differential/                # pruebas con evidencia grabada
├── docs/
│   ├── system-design.md
│   ├── specification.md            # siete secciones, IDs reales y confianza
│   ├── architecture.mmd
│   ├── architecture.svg
│   ├── old-system/                  # enunciado y ejemplos recibidos
│   └── research/                    # estrategia e hipótesis de investigación
├── deploy/
│   └── Dockerfile                   # solo Go; configuración Railway documentada
├── scripts/                         # verificación y generación de entregables
├── experimentos.jsonl              # evidencia registrada; IDs 1–50 en v1
├── artifacts/                      # diferencias y ejecuciones de replay
└── .env                            # local, ignorado; jamás incluido en imágenes
```

SQLite vive en una ruta configurable (`STATE_PATH`) dentro del volumen, por ejemplo `/data/state.db`, no dentro de la imagen ni del repositorio. Archivos auxiliares de SQLite deben quedar en el mismo volumen. `TEAM_TOKEN` se configura en el runtime; `BB_TOKEN` solo en la herramienta de investigación. `.env`, archivos de estado y secretos se excluyen de Git y del contexto de construcción.

Estado, atomicidad y operación

Modelo inicial: cuentas (ID, datos de alta, saldo), operaciones (ID, tipo, datos necesarios para reproducir su efecto y anulación) y contadores de IDs. Su estructura definitiva, unidades monetarias, alcance por token y consumo de contadores se determinan con sondas. Saldos, montos y contadores se representan con enteros `int64`, nunca `float64`. Rechazo de decimales, exponentes, overflow y rango son supuestos pendientes de sondas; no convertir ni truncar valores fuera del rango. Serializar enteros sin sufijo `.0`.

Una única conexión escritora SQLite y serialización de requests con estado, incluidos consultas y reset, proporcionan una vista consistente. Cada comando que modifica estado confirma una transacción antes de responder; reset borra el alcance y reinicia contadores en una transacción. El alcance exacto del reset sigue pendiente de evidencia. Un error de validación no debe mutar estado salvo que el original demuestre lo contrario.

No prometer ejecución exactamente una vez ante cortes de red: una operación puede haberse

confirmado antes de perder la respuesta. No añadir deduplicación o reintentos de mutaciones que alteren la semántica observada. Readiness exige almacenamiento abierto y esquema listo; la respuesta externa de `/health` debe mantenerse compatible con el contrato observado.

Contrato HTTP observado y límites de transporte

Usar un `http.Handler` propio como `Server.Handler`, sin mux, redirects ni páginas de error predeterminadas. Las rutas observadas son exactamente `POST /msg`, `POST /__reset` y `GET /health`. Otros métodos y rutas probados devuelven 404 y `{"error":"E404","mensaje":"ruta desconocida"}` (log 41-44). No limpiar `//msg` ni resolver segmentos `...`. La normalización de escapes, HEAD, OPTIONS y requests rechazados por `net/http` antes del handler requiere sondas: no afirmar cobertura.

Health y reset autenticados devuelven 200 `{"ok":true}` (log 34-35); sin token dan 401 `{"error":"E001","mensaje":"no autorizado"}` (log 36, 50). El chequeo de disponibilidad debe enviar `TEAM_TOKEN`; confirmar que el hosting permite ese header antes de configurar un health check. No añadir una ruta pública de readiness. El supuesto para otros tokens es 401 E001, con estado únicamente para `TEAM_TOKEN`. Auth frente a ruta inexistente o JSON inválido sigue sin precedencia demostrada.

Versionado, captura del evaluador y monitoreo

`BEHAVIOR_VERSION` se lee solo al arrancar, junto con `TEAM_TOKEN` y `STATE_PATH`. Cada versión conserva un conjunto de reglas; `v2` reutiliza `v1` y sobrescribe su delta, con esquema SQLite común. Rollback: configurar `v1` y reiniciar el mismo artefacto sin recompilar; verificar corpus y compatibilidad del estado antes de usarlo. La versión no aparece en rutas ni respuestas. Cada prueba y cada regla cita una `epoch` del explorador.

Capturar cada request del evaluador en `/data/requests.jsonl` y stdout: timestamp UTC, ID local de secuencia, método, ruta y query sin normalización, cuerpo exacto en base64, status, respuesta exacta en base64, duración, versión y regla por defecto utilizada. Un reset autenticado exitoso delimita una secuencia; conservar también los resets fallidos. Omitir `Authorization`, cookies y headers secretos, sin importar sus mayúsculas; registrar solo etiqueta de auth (equipo/otro/ausente) y headers permitidos explícitamente. Si un cuerpo contiene un token configurado, redactarlo antes de cualquier salida y marcar que sus bytes no son reproducibles. Probar ausencia de secretos en JSONL y stdout.

No crear `/logs` ni `/metrics`: acceder por herramientas del hosting. Rotar el archivo a 10 MiB y conservar archivos rotados hasta extraerlos; nunca perder evidencia silenciosamente. El fallo de escritura no cambia status/cuerpo: avisar por stderr y conservar stdout si está disponible. Antes del corte de tokens, extraer secuencias y repetirlas con hipótesis y reset mediante `bb.py`, sin caché, bajo cuota y coordinación exclusiva del token.

Contadores internos: total, status, código de error, defaults, duración máxima y p95 en una ventana de 1.000 requests. Emitir resumen cada 100 requests o minuto y alerta sobre 1 s. Registrar arranque con versión, almacenamiento y esquema listos; nunca secretos. Estas capacidades están diseñadas y siguen pendientes de implementación y verificación.

Investigación y validación

El log 1-50 ya reproduce el corpus y mide precedencia y sobre HTTP en `v1`. Orden observado: E201 antes de E202, y E202 antes de E301/E302/E401 (log 23-32). Priorizar la fórmula del control: descubrir comandos con controles arbitrarios queda bloqueado por E202. Investigar en tandas de hasta 100 solicitudes con hipótesis nuevas. Después cubrir estado, anulaciones, límites, sobre HTTP y secuencias nuevas de 2-12 pasos. La numeración de errores y los nombres bancarios orientan sondas; no prueban reglas ni comandos adicionales.

El cliente exige hipótesis y registra **cada intento**, incluidos reset, health, rutas y autenticación inválida, antes de abrir la conexión. Conserva los campos del README y añade método/ruta, bytes de request y response cuando existe, timestamp, versión de corpus y etiqueta derivada del token sin su valor secreto. El bloqueo del log asigna IDs únicos. Un fallo de transporte permanece como intento sin respuesta y no prueba comportamiento; si ocurre en una secuencia con estado, abortar y repetir desde reset, sin reintento ciego.

El limitador coordina persistentemente entre procesos: 8 req/s, contador por token/día y tope local

conservador configurable de 7.800 (nunca por encima del máximo externo de 8.000), incluyendo resets y reintentos. La reserva de turno y presupuesto es atómica en el mismo área privada del explorador; no contiene el token, solo una etiqueta derivada. Las respuestas `X-Frontier-Infra: rate-limit` se marcan como infraestructura, se excluyen de evidencia y comparaciones y consumen presupuesto local. Aplicar backoff; en secuencias de estado, repetir desde reset cuando sea seguro. La caché usa el prefijo completo desde reset, versión y `Content-Type` efectivo; sus aciertos pendientes persisten para que otra ejecución los reenvíe antes del primer fallo de caché. Se evita para confirmaciones y detección de cambios. El log omite autorización y cualquier aparición de secretos.

Un comando manual para detectar cambios

Interfaz prevista, **todavía no implementada**:

```
python3 tools/explorer/cli.py replay \
  --target original --corpus tests/corpus \
  --compare artifacts/baseline --out artifacts/current
```

La primera ejecución usa `--out artifacts/baseline` sin `--compare`. El comando valida configuración/cuota, hace reset por secuencia, ejecuta en orden, registra intentos y produce un informe por paso con status, cuerpo y efecto observable. Compara JSON sin convertir números a floats ni ordenar arrays, y texto no JSON de forma exacta; además informa diferencias de bytes para cumplir el requisito más estricto de AGENTS.md. No modifica automáticamente la referencia, la especificación ni el servicio. Salida prevista: 0 sin diferencias, 1 con diferencias, 2 ejecución incompleta/error; una ejecución incompleta nunca cuenta como éxito.

Tras un delta: repetir para descartar ruido, diseñar sondas mínimas, citar IDs nuevos, añadir regresiones, cambiar Go y redesplegar conservando el volumen. Validar el corpus anterior y el nuevo; marcar qué reglas fueron reemplazadas. No seleccionar una versión por hora en runtime: configurar la versión vigente a la entrega. Cada fila de la especificación se revalida con IDs nuevos de la `epoch` vigente o se marca histórica/reemplazada; no elevar observaciones de una sola corrida a `confirmado`. El replay incluye auth, rutas y sobre JSON.

Horarios de corte y plan B

El usuario decidió Bogotá UTC-5 en el plan coordinado; falta confirmar con los organizadores la zona oficial. Hasta entonces usar el plazo de entrega más temprano y dos replays tras los posibles cambios. Todos son hitos manuales, sin tarea automática creada.

Hito del 10 de octubre	Bogotá (UTC-5)	CDMX (UTC-6)	UTC
Replay tras posible cambio en Bogotá	15:35	14:35	20:35
Replay tras posible cambio en CDMX	16:35	15:35	21:35
Corte de búsqueda de fórmula; activar plan B si no hay fórmula confirmada	18:00	17:00	23:00
Replay final completo y casos capturados del evaluador	18:30	17:30	23:30
Último despliegue previsto	19:15	18:15	00:15 del 11 de octubre
Congelar código/configuración y validar endpoint	19:30	18:30	00:30 del 11 de octubre
Entrega conservadora	20:00	19:00	01:00 del 11 de octubre

Plan B del control, **supuesto**, no regla del original: usar valores conocidos para mensajes ya registrados; para mensajes nuevos aceptar un campo final de 1-2 dígitos ASCII, 0-99; el resto da E201. No inventar E202 para mensajes nuevos sin fórmula. Mantener separado el comportamiento observado de PING. Este fallback pierde casos de control inválido y puede alterar estado en secuencias: activar solo si falla la investigación y registrar sus usos. Incluir sondas de UTF-8 (bytes frente a caracteres), caja, separadores y ceros iniciales.

Antes de congelar: comprobar créditos, volumen, una réplica y Serverless desactivado, guardar el entrypoint y resultado de la validación; confirmar si el evaluador limita validaciones. Mantener el mismo servicio disponible al menos hasta el día siguiente y hasta conocer el resultado final. La hora exacta de evaluación posterior y los créditos siguen sin confirmar. No redesplegar durante una validación; después

del congelado, solo intervenir ante una avería comprobada, registrar motivo y repetir smoke test y validación pública.

Especificación y entrega de evidencia

La especificación contiene las siete secciones del contrato, con IDs reales del log y confianza por comportamiento. El diseño no es evidencia de implementación.

El log de auditoría conserva todos los intentos y sus IDs originales. Para entregar evidencia, exportar solo entradas `completed`, con status/respuesta y sin marca de infraestructura, conservando esos IDs sin renumerar. Adjuntar aparte los intentos inciertos e infraestructura con su explicación; nunca presentarlos como comportamientos del servicio. Validar que toda cita existe y que ninguna secuencia quedó incompleta. Confirmar con los organizadores si exigen un archivo único de auditoría completo además de la exportación filtrada. El exportador y su validación siguen pendientes; los campos extra no sustituyen los obligatorios.

Plan de implementación y aceptación

1. **Evidencia:** cliente instrumentado y runner; replay de referencia, auth/rutas/reset y precedencia. Cada comportamiento futuro cita IDs del log, sin tokens.
2. **Compatibilidad:** control y gramática, comandos y estado; conservar casos límite y no asumir monotonía de límites antes de aplicar búsqueda binaria.
3. **Persistencia:** SQLite y reset atómico; verificar que reiniciar conserva saldos, IDs y anulaciones, y que un reset restaura exactamente el estado observado.
4. **Regresión y cambio:** secuencias nuevas, errores combinados y replay manual del delta; pruebas locales del corpus tras cada cambio, sin usar el original en el runtime.
5. **Entrega:** Railway con volumen, smoke test público de auth/health/reset/msg/rutas desconocidas, latencia por paso menor de 5 s, corpus vigente y revisión del log/spec.

El diseño inicial no equivale a compatibilidad demostrada. La entrega final exige evidencia para la especificación, pruebas de persistencia y replay público. La evaluación contiene 100 tests (40 individuales, 45 secuencias de 2-5 pasos y 15 de 6-12); reproducir solo los 20 ejemplos no garantiza puntaje alto.

Fuentes de hosting: Railway trial, planes y límites, Serverless y ngrok Free.

Caja negra: especificación incremental

Versión observada: `v1`. Evidencia: `experimentos.jsonl`, IDs 1-50, tres secuencias con reset. El corpus entregado se reprodujo una vez; se conserva confianza **probable** hasta una confirmación independiente. Este documento no afirma vigencia después del cambio. No hay delta observado aún.

Cada fila observada cita IDs reales del log. Las decisiones del reemplazo llevan **supuesto** y no se presentan como evidencia del original. Tras cada cambio, repetir todas las reglas que se entreguen como vigentes y anotar IDs y versión nuevos; retirar de esa entrega las filas históricas que no se revaliden.

1. Entrada

ID	Comportamiento	Evidencia	Confianza
ENT-01	`POST /msg` requiere JSON con `m` string. `{}` y `m` numérico dan E102; campos extra y Content-Type text/plain se aceptan en PING.	log 2-21, 46-49	probable
ENT-02	Sin auth, auth vacía, token inválido o esquema Basic: 401 `{"error":"E001","mensaje":"no autorizado"}`. Health y reset también requieren auth.	log 36-40, 50	probable
ENT-04	Token distinto de TEAM_TOKEN: el reemplazo responderá 401 E001. No se conoce el conjunto de tokens válidos del original.	Decisión; sin prueba	supuesto
ENT-03	GET /health y POST /__reset autenticados: 200 `{"ok":true}`. GET /msg, rutas inexistentes y POST //msg: 404 E404, sin redirect. HEAD, OPTIONS y precedencia auth/ruta pendientes.	log 34-35, 41-44	probable

2. Integridad

ID	Comportamiento	Evidencia	Confianza
INT-01	`ALTA;Lucia;83` da E202 y `ALTA;Lucia;82` crea cuenta. Fórmula y cobertura del control desconocidas.	log 3-4	probable
INT-02	Control 5X en el depósito observado da E201. Rango, espacios y representación pendientes.	log 18	probable
INT-04	Sin fórmula al corte: validar valores conocidos; para mensajes nuevos aceptar control ASCII de 1-2 dígitos 0-99, resto E201. Se omite E202 desconocido; puede alterar estado. No describe al original.	Plan B del diseño; sin prueba	supuesto
INT-03	`ALTA;Lucia;Gomez;11` da E302. E202 precede E302 para controles inválidos probados (log 26-28).	log 21	probable

3. Comandos

Cada fila es el bloque inicial del comando; ampliarlo con argumentos, límites, efectos y errores una vez medidos. No limitar la exploración a estos seis comandos.

ID	Comportamiento	Evidencia	Confianza
CMD-PING	`PING;64` y `PING;1` → 200, `{"pong":true}`; no basta para afirmar que PING ignora cualquier control.	log 2, 33	probable
CMD-ALTA	Alta de Lucia/Mateo → 201, `{"cuentaId":"AC-0001"}` / `AC-0002`; las cuentas observadas arrancan con saldo 0.	log 4-6	probable
CMD-CONSULTA	Consulta devuelve 200 y claves `cuenta_id`, `saldo`; `CONSULTa` funciona en ese caso. Normalización general pendiente.	log 6, 9, 12, 20	probable
CMD-DEPOSITO	Depósitos observados afectan saldo y devuelven 200, `{"status":"recieved","operacionId":"OP-NNNN"}`.	log 7, 19-20	probable
CMD-TRANSFER	Transferencias observadas mueven saldo y devuelven la misma forma de operación; límites no generalizados.	log 8-12	probable
CMD-ANULA	Primera anulación: 200, `{"ok":true,"anulada":"OP-0003"}`. Repetición: 200, `{"ok":false,"motivo":"ya anulada"}`. Reversión y otros tipos pendientes.	log 14-15, 17, 20	probable

4. Memoria entre requests

ID	Comportamiento	Evidencia	Confianza
MEM-01	Saldos, cuentas y operaciones afectan requests posteriores; IDs observados consecutivos. Consumo de IDs ante errores pendiente.	log 4-20	probable
MEM-02	Reset inicia las secuencias observadas; X-Req no se reinicia. Alcance completo e aislamiento por token pendientes.	log 1, 22, 34; contrato	supuesto
MEM-03	Persistir estado tras reinicio es una decisión del reemplazo, no una observación del ciclo de vida del original.	Decisión Q5/Q8; prueba de recuperación pendiente	supuesto

5. Errores

Cuerpos exactos observados; los espacios dentro de strings son significativos. Orden medido: E201 antes de E202, y E202 antes de E301/E302/E401 (log 23-32). El orden entre errores de comando/estado y auth/ruta sigue pendiente.

ID	Comportamiento	Evidencia	Confianza
ERR-001	401, `{"error":"E001","mensaje":"no autorizado"}`.	log 36-40, 50	probable
ERR-100	400, `{"error":"E100","mensaje":"json inválido"}` para cuerpo `{`.	log 45	probable
ERR-102	400, `{"error":"E102","mensaje":"falta m"}` para `{}` y `{"m":64}`.	log 46-47	probable
ERR-404	404, `{"error":"E404","mensaje":"ruta desconocida"}` para rutas/métodos probados.	log 41-44	probable
ERR-201	400, `{"error":"E201","mensaje":"control mal formado"}`.	log 18	probable
ERR-202	400, `{"error":"E202","mensaje":"control inválido"}`.	log 3	probable
ERR-301	400, `{"error":"E301","mensaje":"comando desconocido "}`; espacio final.	log 13	probable
ERR-302	400, `{"error":"E302","mensaje":"aridad inválida"}`.	log 21	probable
ERR-401	404, `{"error":"E401","mensaje":"cuenta inexistente"}`.	log 16	probable

ID	Comportamiento	Evidencia	Confianza
ERR-506	422, `{"error":"E506","mensaje":"límite excedido"}` al transferir 120000 en esa secuencia.	log 10	probable
ERR-507	422, `{"error":"E507","mensaje":"saldo insuficiente"}` al transferir 50000 tras anulación.	log 17	probable

6. Cambios durante el día

ID	Comportamiento	Evidencia	Confianza
CAM-01	Se anuncia un cambio a las 15:30; la evaluación usa la versión vigente a las 20:00. No hay delta observado aún.	AGENTS.md y contrato; replay pendiente	supuesto
CAM-02	Anotar versión/corpus, primer timestamp observado y IDs antes/después; sin sondeo continuo no afirmar el instante exacto del cambio.	Decisión Q6/Q10; informe pendiente	supuesto

7. Riesgos

ID	Comportamiento	Evidencia	Confianza
RIE-01	La anulación observada no equivale a un diseño bancario seguro; investigar negativos, duplicados y operaciones inexistentes antes de describirlos. Un sistema nuevo requeriría reglas explícitas y auditoría.	log 14-17; sondas pendientes	probable
RIE-02	No cambiar mensajes inconsistentes (`recieved`, claves heterogéneas, espacios) durante el reto. Un cliente nuevo usaría un contrato versionado.	log 4, 6-8, 13	probable
RIE-03	Reinicios, volumen mal montado, cuota agotada o pérdida de red pueden hacer fallar evaluación. Verificar recuperación, persistencia y endpoint público antes de entregar.	Decisiones Q5/Q9; pruebas pendientes	supuesto
RIE-04	Un fallback desconocido puede reducir puntaje. No inventar comportamiento silenciosamente: medirlo o documentar explícitamente status/ cuerpo y confianza antes de implementar.	Regla del contrato; fallback explícito en INT-04, sin implementación	supuesto